

SubsFlow — Architecture & Workflow Guide

Enterprise-Grade Multi-Tenant Subscription & Payment Orchestration Engine

Stack: Spring Boot 3, PostgreSQL 16 (RLS), React 18

Architecture: Multi-Tenant RLS & Outbox Pattern

Scope: End-to-End Lifecycle & Engine Flow

1. Executive Summary

SubsFlow is an end-to-end recurring billing, metered usage aggregation, and payment orchestration platform designed for multi-tenant SaaS environments. It combines high-throughput usage ingestion, automated billing and invoice scheduling, resilient payment gateway processing with cryptographic idempotency, automated dunning recovery for failed transactions, and database-enforced Row-Level Security (RLS).

Key Architectural Pillars:

- **Hardware-Strict Tenant Isolation:** Database-level isolation enforced via PostgreSQL Row-Level Security (RLS) and dynamic session variables.
- **Transactional Outbox Pattern:** Zero dual-write discrepancies between DB commits and asynchronous event dispatching.
- **Idempotent Payment Engine:** SHA-256 idempotency locks preventing double-billing on retry storms or gateway latency.
- **Automated Dunning Scheduler:** Intelligent exponential backoff retry queue with graceful suspension handling.

2. Core Modules & System Catalog

Module	Key Java Packages / Classes	Functional Responsibility
Multi-Tenancy & Security	com.subsflow.tenant TenantAuthFilter, TenantContext	Intercepts API keys (X-API-Key), sets DB session variable app.current_tenant_id, and isolates all queries per tenant.
Subscription & Plans	com.subsflow.subscription BillingPlan, SubscriptionService	Manages flat, tiered, and metered plans. Governs subscription lifecycles (ACTIVE, PAST_DUE, CANCELED) and calculates prorated upgrades/downgrades.
Billing & Metering	com.subsflow.billing UsageEvent, InvoiceScheduler	Ingests raw usage telemetry, aggregates consumption across billing windows, applies rate cards, and generates line-item invoices.
Payment Orchestration	com.subsflow.payment PaymentServiceImpl, IdempotencyService	Communicates with payment gateways (sandbox & real), tracks transactions, and locks concurrent charge attempts with SHA-256 keys.
Dunning & Recovery	com.subsflow.dunning DunningScheduler, PaymentRetryQueue	Polls failed payment attempts, executes exponential backoff retries, sends customer notices, and transitions accounts to PAST_DUE on final failure.
Transactional Outbox	com.subsflow.common.outbox OutboxScheduler, OutboxEvent	Guarantees at-least-once message delivery for external webhooks/events within the same atomic database transaction.

3. End-to-End Execution Flow

The SubsFlow engine orchestrates events across multiple asynchronous and scheduled lifecycles. Below is the detailed step-by-step path:

Step 1: Tenant Onboarding & Request Interception

When an incoming HTTP request reaches SubsFlow with header X-API-Key, the TenantAuthFilter validates the credentials, attaches the tenant context to the thread, and executes a database session command SET LOCAL app.current_tenant_id = 'tenant-id'. All subsequent JPA/Hibernate operations automatically filter rows using PostgreSQL RLS policies.

Step 2: Plan Selection & Subscription Creation

The tenant defines billing plans (e.g. \$49/mo flat-rate + \$0.05/API unit). A customer subscription is created with status ACTIVE and defined period boundaries (current_period_start to current_period_end). Mid-cycle tier modifications compute exact proration amounts and credit notes.

Step 3: High-Frequency Usage Event Ingestion

Applications stream metered usage events (such as storage gigabytes, API invocations, or compute runtime) via /api/v1/usage. Events are written to the usage_event table with timestamps and quantity metrics.

Step 4: Invoice Generation & Rating Engine

The InvoiceScheduler continuously monitors billing cycles. Upon period completion, it aggregates all raw usage events, sums the base subscription price plus unit overages, generates line items (invoice_line_item), and issues a final Invoice with status UNPAID.

Step 5: Idempotent Payment Execution

The engine triggers payment processing via PaymentService. An idempotency check (keyed by tenant_id + invoice_id + cycle) verifies that no charge is in progress. The payment gateway is invoked. If successful, the invoice status changes to PAID and the subscription is renewed.

Step 6: Dunning Management on Payment Failure

If the payment fails (e.g. card declined, timeout), the transaction is marked FAILED and immediately queued in payment_retry_queue. The DunningScheduler initiates an exponential backoff sequence (e.g., Retry 1 at +24h, Retry 2 at +72h, Retry 3 at +168h). If all retries fail, the subscription status updates to PAST_DUE or CANCELED.

Step 7: Transactional Outbox Event Publishing

Every state transition (invoice issued, payment completed, subscription canceled) creates an OutboxEvent in the same database transaction. The OutboxScheduler polls pending events and reliably publishes them to webhooks or downstream services.

4. Database Schema & Data Isolation Model

Table Name	RLS Enabled	Primary Key & Foreign Keys	Key Fields
tenant	No (Metadata)	PK: id	name, api_key, status
billing_plan	YES (RLS)	PK: id, FK: tenant_id	name, billing_type, price, billing_period
subscription	YES (RLS)	PK: id, FK: plan_id, tenant_id	status, current_period_start, current_period_end
usage_event	YES (RLS)	PK: id, FK: subscription_id	quantity, event_type, timestamp
invoice	YES (RLS)	PK: id, FK: subscription_id	amount, status, due_date
payment_transaction	YES (RLS)	PK: id, FK: invoice_id, subscription_id	amount, status, idempotency_key, gateway_ref
payment_retry_queue	YES (RLS)	PK: id, FK: transaction_id	retry_count, next_retry_at, status
outbox_events	YES (RLS)	PK: id, FK: tenant_id	event_type, payload, status (PENDING/SENT)
idempotency_key	YES (RLS)	PK: (tenant_id, key)	request_hash, status, response_payload

5. Interactive Frontend Dashboard & Sandbox

SubsFlow includes a full-featured React 18 UI designed for visual observability, operational management, and real-time failure simulation:

UI Component	File Path	Feature & Operational Capability
Pipeline Visualizer	PipelineVisualizer.jsx	Live visual node map illustrating subscriptions flowing through Ingestion, Rating, Invoicing, Payment, Dunning, and Outbox stages.
Gateway Sandbox	GatewaySandbox.jsx	Interactive fault injection panel to simulate gateway card declines, network timeouts, invalid cvc errors, and verify dunning response in real time.
Subscription & Plan Manager	SubscriptionsPanel.jsx PlansPanel.jsx	Form for subscribing customers, switching tiers with immediate proration breakdown, and setting up custom usage pricing models.
Invoices & Usage Panel	InvoicesPanel.jsx UsageForm.jsx	Emits synthetic telemetry usage events and lists issued invoices with itemized line breakdowns and real-time payment retry histories.
API & Outbox Log Stream	ApiLog.jsx	Inspects real-time HTTP traffic, tenant context headers, latency benchmarks, and outbox event payload dispatches.

6. Reliability & Production Readiness Checklist

- **PostgreSQL Row-Level Security (RLS):** Zero risk of cross-tenant data leaks even if developer omits WHERE tenant_id = ? in JPA queries.
- **Optimistic Locking:** Uses @Version on Subscription and BillingPlan to prevent lost updates during concurrent plan migrations.
- **Idempotency Guarantees:** Payment requests are hash-validated against the idempotency_key table to eliminate double charging.
- **At-Least-Once Delivery:** Transactional Outbox pattern guarantees that event bus crashes or external webhook timeouts never drop domain events.
- **Metrics & Observability:** Prometheus meters track active subscriptions, dunning failure counts, billing run duration, and payment success rates.